

Week- 4

Backend Engineering

Pawan Dhanjwari

Launchpad

UNIT TESTING AND DEBUGGING

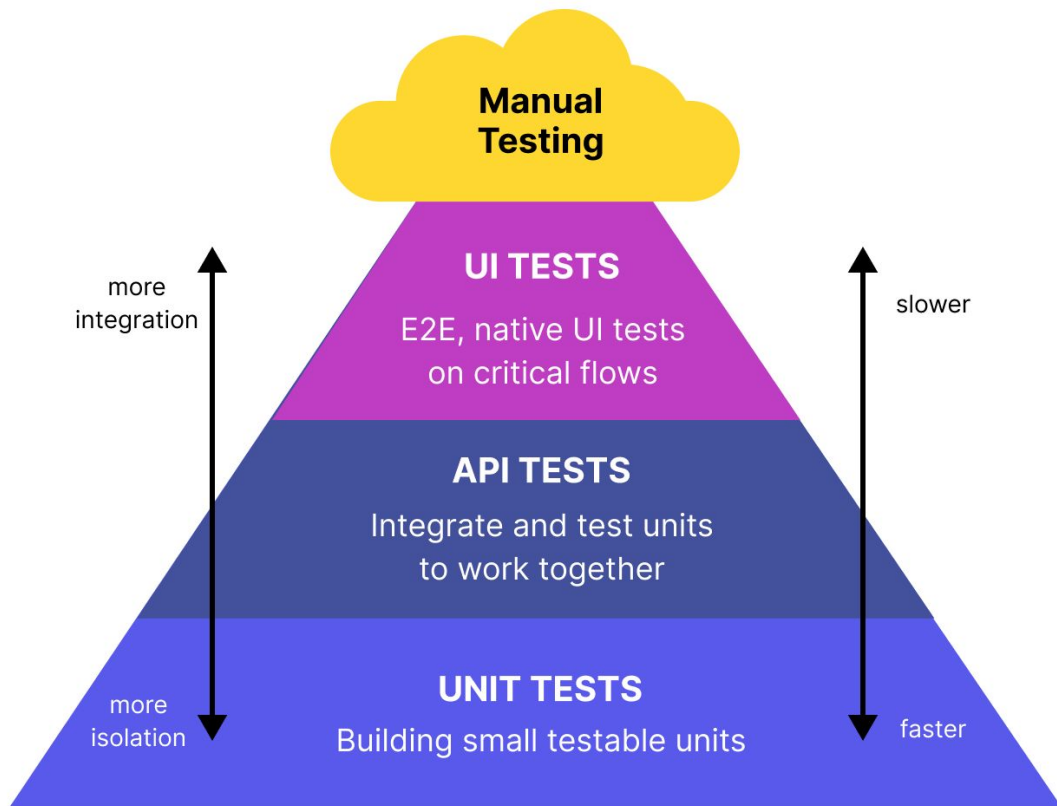
Agenda

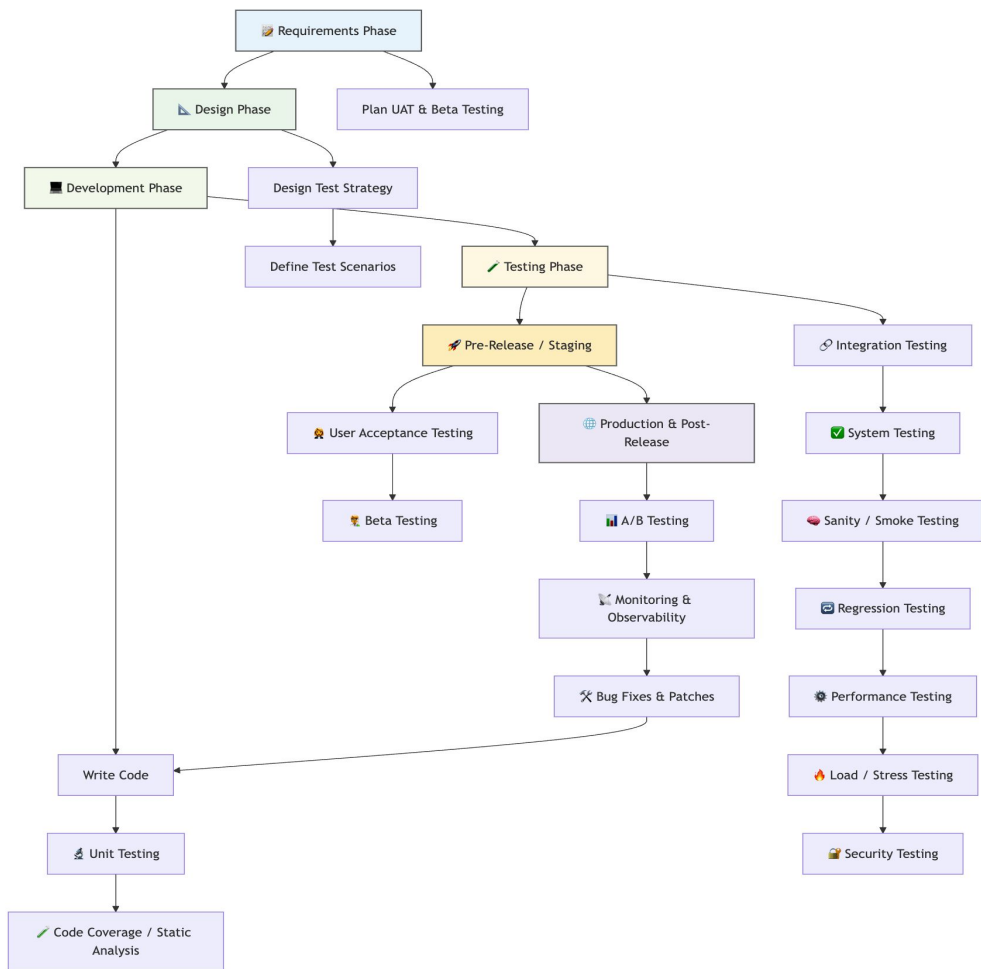
- Why do we need unit testing ?
- Frameworks for unit testing in Java
- Writing effective unit testing - Understanding mocking code coverage
- Continuous Integration - Integrating unit testing into continuous integration
- Unit testing exercise for Learner Management System
- Question and answers

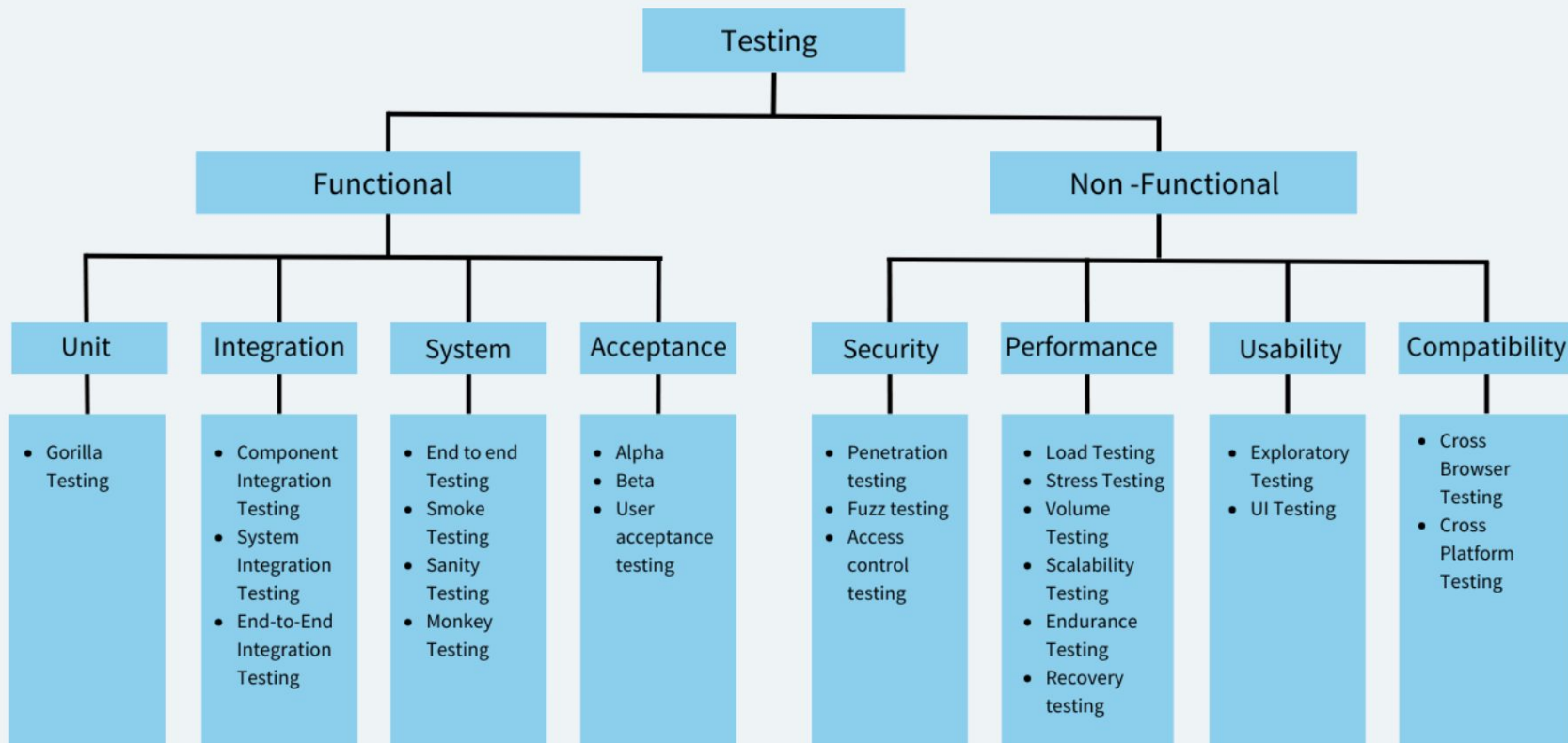
What are unit tests and why test ?

- Unit tests help catch errors early in the development process
- Helps improving code quality
- Allow safe refactoring and modularization of code
- Encourages collaboration and allows parallel development
- Tests serve as good documentation
- **Tests help reduce fear of things breaking on prod :)**

Unit Vs Integration Testing







Difference between unit and integration testing

Unit Tests

- ▶ Isolate each part of the program

- ▶ Show that the individual parts are working correctly

Integration tests

- ▶ Test the interoperability of multiple subsystems

- ▶ Tests that “the nuts fit the bolts”

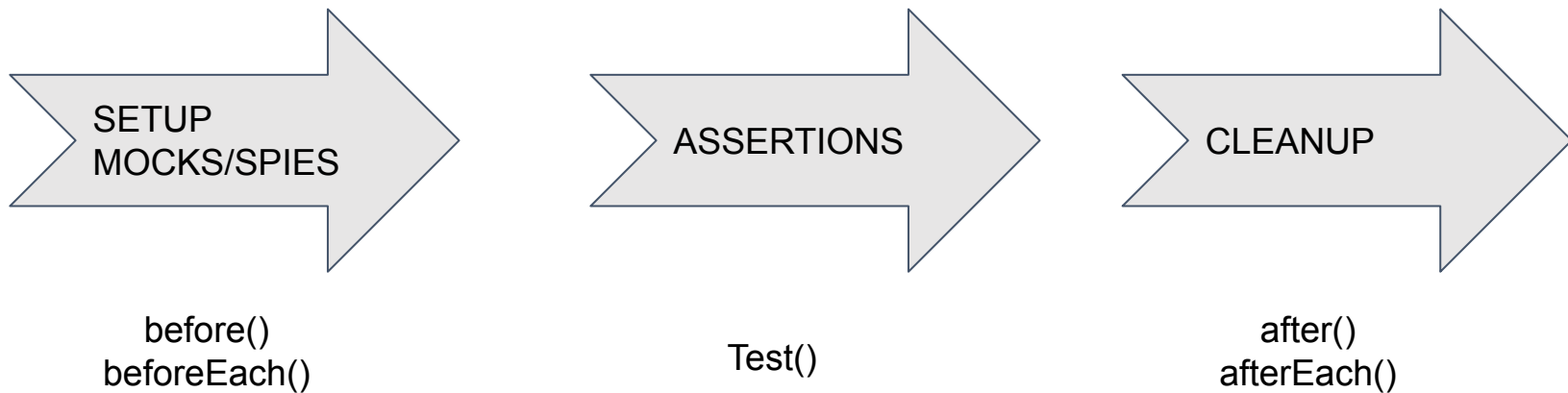
Frameworks for unit testing – Java

- Unit testing frameworks - junit, testng
- Mocking frameworks - Mockito
- Assertion framework - assertj
- Integration test framework - SpringBootTest

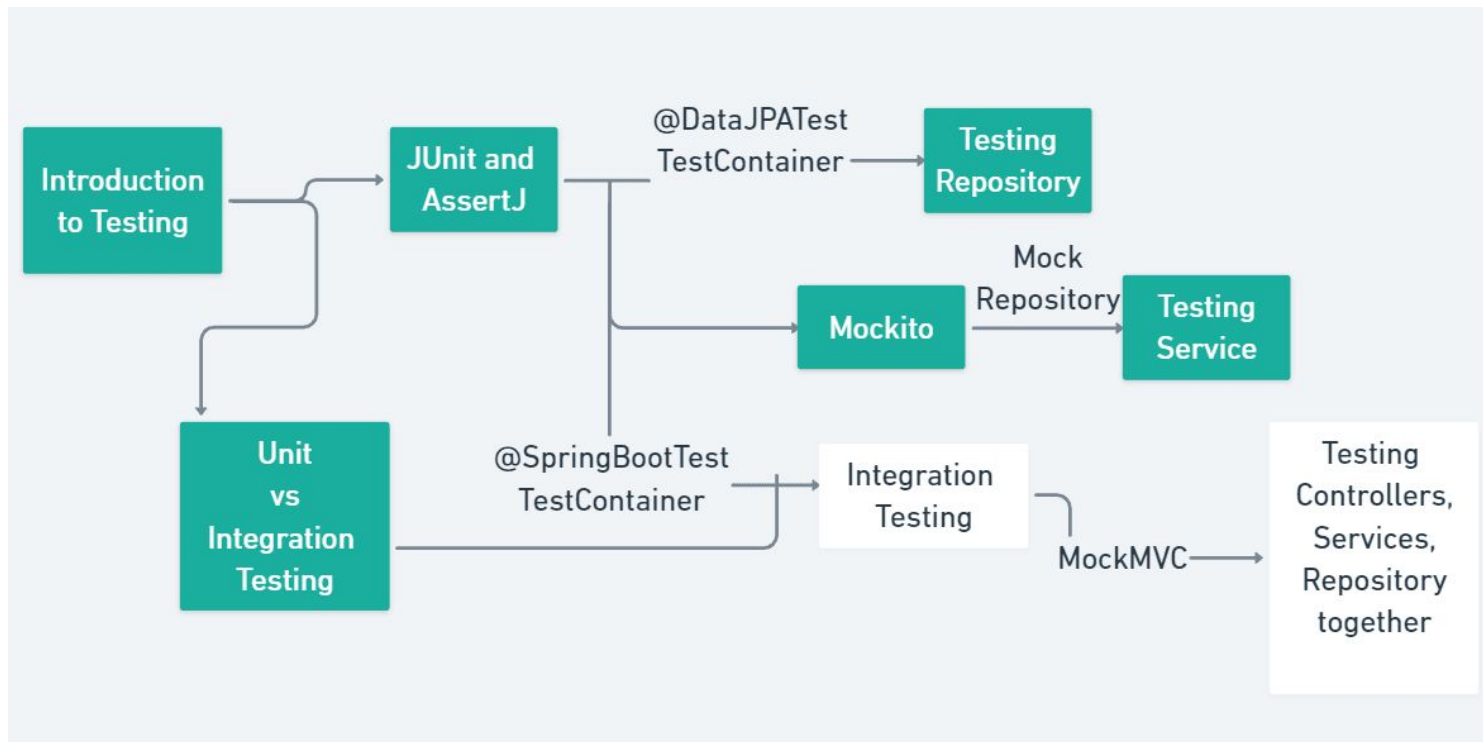
Anatomy of unit testing



Anatomy of unit testing – JUnit



Unit testing Spring Boot



Unit testing Spring Boot

What Are We Testing?

You are testing the **business logic** of your application, **not** database interactions.

- **Service Layer** = contains the core business logic.
- **Repository Layer** = interacts with the database (external system).

In a **unit test for the service layer**, we want to test the logic **in isolation**, so we replace (mock) the repository.

Unit testing Spring Boot

Analogy:

Think of the Repository as a vending machine that gives snacks (data) when you press a button.

You don't care how the machine works internally during your logic test — you **mock it** to just give a snack when asked.

Best Practices Unit Testing

Isolate Dependencies

- Focus on testing a specific unit of code in isolation.
- Mock or stub external dependencies like databases, APIs, or modules.
- Tools: Mockito

Independent and Isolated Tests

- Each test should be independent and not rely on others.
- Isolating tests prevents failures from cascading.

Descriptive Test Names

- Use clear and descriptive names for test cases.
- Reflect the behavior or functionality being tested.
- Enhances readability and understanding of test failures.

Arrange Act Assert Patterns

- Structure tests into three sections: Arrange, Act, Assert.
- Arrange: Set up preconditions.
- Act: Execute the code being tested.
- Assert: Verify expected behavior and results.

Positive And Negative Scenarios

- ▶ Test both expected positive scenarios and edge cases.
- ▶ Identify and handle error conditions or unexpected behavior.

Use Assertions

- ▶ Utilize assertion libraries
- ▶ Verify return values, exceptions, and side effects.

Test Coverage

- Employ code coverage tools
- Measure the effectiveness of your tests.
- Aim for high test coverage.

Readable and Maintainable Tests

- Write clean, readable, and maintainable test code.
- Follow coding conventions and use descriptive names.
- Apply the DRY principle.

Automation

- Incorporate unit tests into an automated testing framework.
- Use continuous integration (CI) systems like Jenkins, Travis CI, or GitHub Actions.
- Catch issues early and ensure consistent execution.

Review and Update

- Regularly review and update tests as your codebase evolves.
- Maintain the accuracy and relevance of your test suite.

Got Questions?

Thank you